Projet final

Supervision d'équipements et suivi d'incidents

Édition 2026-2027 · Développement Full Stack avec Java (ALTN72)

À rendre avant le	**5 novembre 2026 à 23 h 59** sur Moodle (voir §11)
Ce que vous devez livrer	Une application web full stack Spring Boot, ses tests automatisés, un script ou une procédure d'initialisation de la base de données et un fichier `journal.md` (voir §8).
Ce qui est attendu de tous	Le **socle fonctionnel** (§5) + au moins **deux fonctionnalités transverses** de votre choix (§6) + le respect des **exigences techniques** (§7), dont le Clean Code, la gestion des exceptions et les principes SOLID.

PLAN DU DOCUMENT

§1 à §4	Contexte, objectifs pédagogiques, utilisateurs, données à gérer
§5	Fonctionnalités attendues (socle commun à tous les groupes) et règles de gestion
§6	Fonctionnalités transverses (lot alpha) : au moins deux à réaliser
§7	Exigences techniques : pile, architecture, Clean Code, exceptions, SOLID, persistance, sécurité, tests, déploiement, Git
§8 à §12	Livrables, évaluation, contrôles, dépôt, nom du projet
Annexes	A. Modèle de journal.md · B. Liste de contrôle avant dépôt · C. Références

1. OBJECTIF

Une entreprise exploite un réseau de transport composé de stations dans lesquelles sont installés différents équipements : ascenseurs, escaliers mécaniques, portiques, distributeurs de titres de transport, écrans d'information, etc.

Lorsqu'un équipement tombe en panne, un incident est déclaré. L'entreprise doit pouvoir suivre cet incident, mesurer depuis combien de temps il est ouvert, l'affecter à un prestataire, enregistrer sa résolution et évaluer ce prestataire.

Votre mission consiste à réaliser une application **full stack** permettant de gérer les équipements et les incidents, mais surtout de fournir aux responsables d'exploitation une **vision claire de la situation**.

2. OBJECTIFS PEDAGOGIQUES

Ce projet doit vous permettre de mobiliser les principaux acquis du cours :

- ✓ Organiser une application Spring Boot multicouche répondant à un besoin métier.
- ✓ Réaliser le backend et le frontend de cette application.
- ✓ Exposer et exploiter des ressources au moyen d'une API REST.
- ✓ Mettre en œuvre la persistance avec Spring Data JPA.
- ✓ Utiliser des modules appropriés de l'écosystème Spring, en fonction de ce que vous souhaitez accomplir.
- ✓ Écrire un code propre (Clean Code), gérer correctement les exceptions et appliquer les principes SOLID.
- ✓ Vérifier votre solution grâce à des tests automatisés.

- ✓ Travailler en équipe avec Git et rendre votre progression observable.
- ✓ Justifier vos choix techniques et fonctionnels.
- ✓ Faire preuve d'autonomie, d'initiative et de créativité.

3. UTILISATEURS ET PROFILS

Deux profils utiliseront l'application. À vous de déterminer ce que chaque profil pourra faire.

L'un des deux profils est le **responsable d'exploitation** : c'est lui qui doit disposer d'une vision claire de la situation. Le second profil est à définir par votre groupe (par exemple un agent chargé de signaler les pannes). Les deux profils s'authentifient via l'écran de connexion décrit en F1.

> **À faire dès le début**
> Complétez le tableau de répartition des droits du §5.6 et recopiez-le dans la section **CHOIX TECHNIQUES** de votre `journal.md`. Une restriction doit être appliquée **côté serveur** : masquer un bouton ne suffit pas.

4. DONNEES A GERER

L'application doit au minimum gérer les stations, les équipements installés dans ces stations, les incidents associés aux équipements, les prestataires susceptibles de prendre en charge un incident, les évaluations de leurs interventions et les utilisateurs.

Élément	Informations minimales
Station	Un identifiant, un nom.
Équipement	Un identifiant · une référence unique · un libellé ou une description · un type · une station · un état de fonctionnement.
Incident	Un identifiant · l'équipement concerné · une description · une date et une heure d'ouverture · un statut · un prestataire éventuel · une date et une heure de résolution éventuelles.
Prestataire	Un identifiant, un nom, de quoi le contacter.
Évaluation	L'incident concerné, une note, un commentaire éventuel, une date.
Utilisateur	Un identifiant de connexion, un mot de passe (jamais stocké en clair), un prénom, un profil.

Le groupe peut enrichir ce modèle si ses choix sont cohérents avec le besoin.

Données de démonstration. Votre application doit démarrer avec des données suffisamment variées pour être évaluée : plusieurs stations, plusieurs types d'équipements, et des incidents ouverts, affectés et résolus, avec des dates différentes. Pensez à couvrir les cas limites des indicateurs d'urgence (§5.5). Comme l'indicateur dépend de l'heure courante, générez les dates d'ouverture **relativement à l'instant présent** (par exemple « il y a 47 heures ») plutôt qu'avec des dates figées.

5. FONCTIONNALITES ATTENDUES (SOCLE COMMUN)

Toutes les fonctionnalités de cette section sont attendues de **tous** les groupes. Elles sont numérotées (F1, F2…) pour que vous puissiez les référencer dans vos tickets, vos commits, vos tests et votre journal.md.

5.1 Accès à l'application

N°	Fonctionnalité	Précisions
F1	Connexion et déconnexion	Ouverture d'une session via un écran de connexion permettant de saisir le login et le mot de passe : • Réaffichage de la même page en cas d'erreur de connexion. • Message d'erreur si les informations de connexion ne sont pas valides. • Une fois la connexion validée, affichage en haut à droite de chaque page de : **Bonjour [Prénom] ([profil]) | Déconnectez-vous**. • Un clic sur « Déconnectez-vous » provoque la fermeture de la session et l'affichage de la page de connexion.

5.2 Équipements

N°	Fonctionnalité	Précisions
F2	Consulter les équipements par station	Retrouver les équipements d'une station, avec leur type et leur état de fonctionnement.
F3	Consulter la fiche d'un équipement	Informations de l'équipement **et** liste de ses incidents (en cours et passés).

5.3 Cycle de vie des incidents

N°	Fonctionnalité	Précisions
F4	Déclarer un incident	Pour un équipement donné. Les formulaires valident les données indispensables et affichent des messages compréhensibles.
F5	Consulter le détail d'un incident	Description, équipement, station, durée d'ouverture, prestataire éventuel, évaluation éventuelle.
F6	Modifier les informations autorisées	À vous de définir, et de justifier, ce qui est modifiable et par quel profil.
F7	Affecter ou réaffecter un prestataire	Un incident non résolu peut être affecté, puis réaffecté.
F8	Déclarer l'incident comme résolu	Enregistre la date et l'heure de résolution.
F9	Consulter l'historique des incidents résolus	Présenté du plus récent au plus ancien.

5.4 Évaluation des prestataires

N°	Fonctionnalité	Précisions
F10	Saisir une évaluation de l'intervention d'un prestataire	Possible uniquement une fois l'incident résolu (voir RG8). La note du prestataire doit pouvoir être consultée quelque part de pertinent (fiche prestataire, historique, fiche équipement…).

5.5 Tableau de bord de supervision

N°	Fonctionnalité	Précisions
F11	Tableau de bord des incidents actifs	Le responsable d'exploitation doit pouvoir : visualiser les incidents actifs et leur niveau d'attention ; repérer leur station et l'équipement concerné ; accéder rapidement au détail d'un incident ; filtrer ou organiser l'affichage selon **au moins deux critères** utiles ; consulter quelques indicateurs synthétiques pertinents.

Le tableau de bord présente **uniquement les incidents non résolus**. Il doit permettre d'identifier immédiatement leur situation.

Pour un incident qui n'est encore **affecté à aucun prestataire** :

Durée depuis l'ouverture	Indicateur
Moins de 48 heures	**Vert**
De 48 heures incluses à moins de 72 heures	**Orange**
72 heures ou davantage	**Rouge**

Un incident **affecté** à un prestataire doit être visuellement distingué des incidents encore en attente d'affectation. Il ne doit pas conserver un indicateur laissant croire qu'aucune prise en charge n'a eu lieu.

L'indicateur doit être calculé à partir de la date et de l'heure **courantes**. Il ne doit pas être enregistré comme une valeur figée dans la base de données.

5.6 Répartition des droits par profil

Recopiez ce tableau dans votre journal.md et indiquez, pour chaque fonctionnalité, ce que chaque profil peut faire (accès complet, lecture seule, interdit). Nommez vos deux profils dans les en-têtes de colonnes.

N°	Fonctionnalité	Profil 1 : …	Profil 2 : …
F2–F3	Consulter équipements et fiches		
F4	Déclarer un incident		
F5–F6	Consulter / modifier un incident		
F7	Affecter ou réaffecter un prestataire		
F8	Déclarer un incident résolu		
F9	Consulter l'historique		
F10	Évaluer un prestataire		
F11	Tableau de bord		

5.7 API REST

N°	Fonctionnalité	Précisions
F12	Exposer une API REST	Elle doit permettre de **consulter** les incidents et de réaliser au moins une opération de **modification de leur état**. Les réponses utilisent des codes HTTP adaptés ; la demande d'une ressource inexistante est traitée proprement. L'API est documentée avec Swagger / OpenAPI (ou un outil équivalent).

5.8 Règles de gestion

Ces règles sont la référence de ce qu'on attend de votre logique métier. Elles doivent vivre dans la couche service (ou dans le modèle), jamais dans les contrôleurs ni dans les vues.

Règle	Énoncé
RG1	Un incident déclaré est rattaché à un équipement, possède une date et une heure d'ouverture et démarre sans prestataire.
RG2	Un incident est soit non affecté, soit affecté à un prestataire, soit résolu.
RG3	Pour un incident non affecté et non résolu : vert en dessous de 48 h, orange de 48 h incluses à moins de 72 h, rouge à partir de 72 h (heure courante moins heure d'ouverture). Un incident ouvert depuis 47 h est donc vert ; à 48 h pile, il devient orange.

Règle	Énoncé
RG4	Un incident affecté possède un état de présentation distinct : il ne porte jamais l'indicateur vert, orange ou rouge de RG3.
RG5	Un incident résolu possède une date et une heure de résolution, postérieures ou égales à son ouverture. Il ne peut plus être affecté ni réaffecté.
RG6	Un incident résolu n'apparaît plus parmi les incidents actifs du tableau de bord, mais reste dans l'historique.
RG7	L'historique des incidents résolus est ordonné du plus récent au plus ancien (par date de résolution). La liste des incidents d'un équipement l'est selon la date d'ouverture.
RG8	Une évaluation concerne un incident résolu et affecté à un prestataire ; il n'y en a qu'une par incident ; la note est un entier de 1 à 5.
RG9	La référence d'un équipement est unique.
RG10	Une opération invalide ne laisse jamais les données dans un état incohérent : toute opération qui modifie plusieurs données est atomique (propriétés ACID).
RG11	Les restrictions liées aux profils sont appliquées côté serveur.

6. FONCTIONNALITES TRANSVERSES (LOT ALPHA)

6.1 Principe

Le socle du §5 est commun à tous. Le **lot alpha** est le terrain de votre singularité : il vous permet de construire une application qui ne ressemble pas à celle des autres groupes, d'explorer des aspects techniques ou fonctionnels et de démontrer votre capacité d'initiative et votre créativité.

> **Règle du jeu**
>
> **Chaque groupe réalise au moins deux fonctionnalités transverses**, choisies dans la liste ci-dessous ou proposées à l'enseignant.
>
> Le contenu du lot alpha doit être choisi **dès le début** du projet et développé **progressivement**. Il ne doit ni être découvert ni être entièrement réalisé pendant la dernière séance.
>
> L'intégration du lot alpha ne doit pas fragiliser les fonctionnalités du socle.

6.2 Choix des fonctionnalités transverses

	Piste	Idée	Pistes techniques
1	**Supervision avancée**	Recherche multicritère, filtres combinables, tris, vues personnalisées, analyse temporelle des incidents.	Requêtes JPQL / Specifications, pagination
2	**Visualisation des données**	Graphiques pertinents, tendances, répartition par station ou par type d'équipement, évolution des délais de résolution, note moyenne des prestataires.	Requêtes d'agrégation, bibliothèque de graphiques
3	**Aide à l'affectation**	Compétences des prestataires, disponibilité, historique des interventions ; proposition **argumentée** d'un prestataire adapté (par exemple les trois mieux notés pour ce type d'équipement).	Service de recommandation, API REST
4	**Cartographie du réseau**	Représentation des stations et de leur situation ; navigation depuis la carte vers les équipements et incidents.	Coordonnées en base, bibliothèque cartographique

	Piste	Idée	Pistes techniques
5	**Journal métier**	Historique détaillé et horodaté des changements importants apportés à un incident (qui, quand, quoi).	Entité d'événements, événements Spring
6	**Interface enrichie par l'API REST**	Mise à jour partielle de l'interface et échanges asynchrones avec le backend (par exemple : choisir un prestataire dans une fenêtre modale sans recharger la page).	JavaScript (fetch), fragments Thymeleaf
7	**Qualité renforcée**	Stratégie de validation, tests d'intégration (contrôleurs, repositories), mesure de la couverture, intégration continue, amélioration argumentée de la conception.	MockMvc, @DataJpaTest, Testcontainers, JaCoCo, GitHub Actions
8	**Source de données externe**	Un système externe (simulateur de capteurs, application mobile fictive…) déclare des incidents via l'API REST de votre application.	Endpoint sécurisé, client de test
9	**Import / export de données**	Import de stations ou d'équipements, export de l'historique (CSV, par exemple).	Spring MVC, Apache Commons CSV

Vous avez une autre idée ? Soumettez-la à l'enseignant **avant** de vous lancer.

6.3 Évaluation du lot alpha

L'évaluation du lot alpha porte d'abord sur la **qualité, la pertinence, l'intégration et la maîtrise** des deux fonctionnalités obligatoires. Un groupe qui réalise davantage de fonctionnalités ne sera donc pas automatiquement mieux évalué. En revanche, toute réalisation supplémentaire du lot alpha, pleinement fonctionnelle, correctement testée, documentée et introduite sans régression pourra donner lieu à une valorisation de la note.

Présentez vos choix, votre progression, vos essais, vos résultats et vos limites dans la section **LOT ALPHA** de votre journal.md.

7. EXIGENCES TECHNIQUES

7.1 Pile technique

1. L'utilisation de **Spring** (Boot, MVC, Data JPA…) est obligatoire.
2. La pile technique de votre application doit être *a minima* identique à celle du **TP fil rouge** (versions de Java et de Spring Boot comprises).
3. Vous souhaitez construire la Vue avec une technologie différente de Thymeleaf ? C'est possible et autorisé. Mais attention : ce choix implique que vous serez en **autonomie totale** sur cette technologie. Vous devrez donc assumer ce choix de bout en bout.
4. Vous souhaitez utiliser une autre base de données que MariaDB ? Là aussi, c'est possible. Seule contrainte : choisissez une base de données **relationnelle**.
5. L'application est un projet **Maven** qui peut être buildé et lancé en ligne de commande.

7.2 Architecture

L'architecture doit faire apparaître des **responsabilités clairement séparées** : contrôleurs, services (règles métier), repositories (accès aux données), modèle. Les règles métier ne sont pas dispersées dans les contrôleurs ni dans les vues. Les choix supplémentaires (bibliothèques, patterns, découpage en packages) sont autorisés s'ils servent le projet et restent compréhensibles par **tous les membres du groupe**.

7.3 Clean Code

Votre code sera lu, relu et évalué : écrivez-le pour un lecteur humain. Suivez les consignes de la fiche *ALTN72 - Clean code - Nommage* et, en particulier :

- ✓ Des **noms** révélateurs, issus du vocabulaire du domaine (équipement, incident, prestataire…), dans une **seule langue** cohérente pour les identifiants .
- ✓ Des méthodes **courtes** qui font **une seule chose** .
- ✓ Pas de « nombres magiques » : les seuils de 48 h et de 72 h sont des constantes nommées, définies **en un seul endroit** .
- ✓ Pas de code mort, pas de code commenté, pas d'import inutilisé, pas de duplication .
- ✓ Des commentaires qui expliquent le **pourquoi**, jamais ce que le code dit déjà .
- ✓ Un formatage homogène (formatage automatique de l'IDE) et les conventions Java habituelles.

7.4 Gestion des exceptions

1. Suivez les consignes contenues dans la fiche *ALTN72 - Clean code - Gestion des exceptions*.
2. Entre *@ControllerAdvice* et *ResponseStatusException*, choisissez l'option qui vous semble la plus pertinente, cohérente et efficace pour gérer les exceptions. Justifiez ce choix dans votre journal.md.
3. Dans tous les cas, vous devez créer des **exceptions personnalisées** avec des messages (en français) pertinents.
4. Les erreurs sont traduites vers le bon code HTTP (par exemple : ressource inexistante, donnée invalide, transition d'état interdite) et, côté pages web, vers un message compréhensible pour l'utilisateur. Aucune trace technique n'est affichée à l'écran.
5. Si et seulement si l'utilisation de *ResponseEntity* vous paraît pertinente, alors exploitez cette classe et soyez prêts à expliquer ce choix.

7.5 Principes SOLID

- ✓ Respectez **au moins un** des cinq principes SOLID et documentez-le dans votre journal.md : où, comment, et ce que cela change concrètement.
- ✓ Pour chacun des principes que vous n'appliquez pas, **justifiez la non-pertinence** de sa mise en œuvre dans votre projet. Une justification honnête vaut mieux qu'une application artificielle.
- ✓ Appuyez-vous sur les supports *ALTN72 - Clean code - SOLID* et *Responsabilité unique*.

7.6 Persistance

1. Votre interface *JpaRepository* ne doit pas être vide. Utilisez un mélange de requêtes **prédéfinies** (dérivées du nom des méthodes) et de requêtes **explicites en JPQL**.
2. Utilisez **une seule fois** une requête SQL standard (native). Dans votre journal.md, faites une analyse critique de cette façon de faire.
3. Si vous utilisez l'annotation *@Transactional*, faites-le de manière pertinente et soyez prêts à justifier chaque utilisation.
4. Fournissez un script SQL ou un mécanisme équivalent qui permet de recréer la base et ses données de démonstration en quelques secondes.

7.7 Sécurité

- ✓ Les mots de passe sont **hachés** (jamais stockés en clair, ni dans la base, ni dans le dépôt Git).
- ✓ Aucun secret (mot de passe de base, clé) n'est écrit en dur dans le code ou versionné : utilisez des variables d'environnement ou un profil de configuration non versionné.

✓ Les pages et l'API sont protégées : un utilisateur non connecté est renvoyé vers la page de connexion (RG11).

7.8 Tests automatisés

Les tests doivent vérifier des **comportements métier utiles** et contribuer à détecter les régressions. Ils s'exécutent avec `mvn test` ou `mvn verify`. Suggestions de règles à vérifier dans vos tests :

- Un incident non affecté ouvert depuis 47 heures est vert.
- À partir de 48 heures, il devient orange.
- À partir de 72 heures, il devient rouge.
- Un incident affecté possède un état de présentation distinct.
- Un incident résolu n'apparaît plus parmi les incidents actifs.
- L'historique est ordonné du plus récent au plus ancien.
- La demande d'un incident inexistant produit la réponse HTTP prévue.

Astuce : pour tester des seuils horaires, injectez l'horloge (`java.time.Clock`) dans votre service au lieu d'appeler `LocalDateTime.now()` partout.

7.9 Déploiement

Déployez votre application sur un serveur de votre choix. Elle sera ainsi accessible via une **URL**, et ce, à partir de n'importe quel ordinateur relié à Internet. Indiquez cette URL et les comptes de test dans le journal.md.

7.10 Git et traçabilité

✓ Le code est versionné avec **Git** dans **un seul dépôt par groupe**.

✓ Chaque membre du groupe **commit** régulièrement ; l'historique raconte le projet.

✓ Le dépôt est accessible à l'enseignant dès la 2e séance PRJ (dépôt privé avec invitation, ou public).

8. LIVRABLES ET FICHIER JOURNAL.MD

La livraison finale comprend au minimum :

- Le code source de l'application .
- Les scripts ou instructions nécessaires concernant la base de données .
- Les tests automatisés, intégrés au projet .
- Un fichier **journal.md**, placé à la racine du projet.

L'enseignant doit pouvoir récupérer votre livraison et lancer votre application **sans se poser la moindre question**. Le journal.md regroupe au minimum les sections suivantes (un modèle est fourni en annexe) :

Section	Contenu attendu
README	Prérequis, configuration, création ou initialisation de la base de données, commande de lancement, comptes et données utiles à la démonstration, URL de l'application déployée, limites connues.
ARCHITECTURE	Lien vers le dépôt, schéma de l'architecture de votre solution.
TESTS	Stratégie de tests, tests unitaires, commande `mvn test` ou `mvn verify`, comportements couverts, comportements qui ne le sont pas encore et explications éventuelles.
CHOIX TECHNIQUES	Principales décisions de conception et justification synthétique : droits par profil, stratégie d'exceptions, principes SOLID appliqués ou écartés, analyse critique de la requête SQL native, usages de @Transactional.
LOT ALPHA	Objectifs, progression, essais, résultats et limites des fonctionnalités transverses choisies.

Section	Contenu attendu
BILAN	Contribution de chaque membre du groupe, plus grande difficulté rencontrée et manière de la surmonter, fonctionnalités non réalisées et pourquoi, trois points à retenir du cours et de ce projet.

9. CRITERES D'EVALUATION

L'évaluation portera principalement sur :

- ✓ La mise en œuvre des points vus en cours .
- ✓ Le fonctionnement de votre application .
- ✓ La justesse de la mise en œuvre des règles métier .
- ✓ La conformité aux principes de **Clean Code** .
- ✓ Le respect d'**au moins un principe SOLID** (à documenter dans votre journal.md) et la justification de la non-pertinence de la mise en œuvre des autres .
- ✓ La qualité des **fonctionnalités transverses** réalisées (au moins deux) .
- ✓ La pertinence, la cohérence, l'ergonomie et la convivialité des écrans du frontend .
- ✓ La capacité à vérifier la solution (richesse et pertinence de la couverture de tests) .
- ✓ La régularité et la traçabilité du travail .
- ✓ L'autonomie du groupe.

L'effet visuel ne compensera pas un socle défaillant. Inversement, une application techniquement correcte mais difficile à comprendre ou à utiliser ne sera pas considérée comme pleinement aboutie.

Bonus et malus

- **Contrôles oraux** : ils pourront produire un bonus, rester neutres ou entraîner un malus. Leur effet cumulé sera compris entre **−2 et +2 points** sur la note finale (sur 20). Ils visent à valoriser un travail régulier, autonome et réellement collectif.
- **Audace et excellence** : l'audace, l'originalité, la capacité à pousser jusqu'au bout la conformité aux bonnes pratiques, la prise de hauteur par rapport au code (architecture et patterns) et les approches surprenantes (dans le bon sens) seront récompensées par un bonus, dans la limite de +2 pts.

10. CONTROLES PAR L'ENSEIGNANT

L'enseignant pourra, à tout moment, à partir de la 2e séance PRJ :

- Récupérer la dernière version de votre dépôt .
- Lancer les tests .
- Demander une démonstration ciblée .
- Interroger au hasard n'importe quel groupe sur une partie du code ou sur un choix réalisé.

Tous les membres du groupe doivent donc comprendre le fonctionnement général de la solution. Une répartition des tâches est normale ; l'existence de zones du projet entièrement incompréhensibles pour l'un des membres ne l'est pas.

11. DEPOT MOODLE ET DATE LIMITE

Dans l'espace dédié au dépôt sur Moodle, chaque groupe dépose, au choix :

- Un fichier texte contenant l'URL de son dépôt sur le repository de son choix (nommez-le de façon à ce que l'enseignant puisse l'identifier sans ambiguïté, par exemple *ALTN72_projet_NOM1_NOM2.txt*) .

- Un fichier .zip contenant tous les fichiers attendus (incluez le dossier `.git` pour que l'historique reste consultable).

Un seul et unique dépôt par groupe, s'il vous plaît !

La démonstration finale devra raconter un usage cohérent de l'application. Elle ne consistera pas à parcourir mécaniquement toutes les pages ni à lire le code à l'écran.

DATE LIMITE POUR DÉPOSER VOTRE PROJET

5 novembre 2026 à 23 h 59

12. UN NOM POUR VOTRE PROJET

Chaque groupe est vivement encouragé à trouver un **nom original** pour son projet, et pas uniquement pour l'entreprise fictive. Vous pouvez également construire une identité visuelle simple autour de ce nom (logo, couleurs).

Le nom de projet jugé le plus original sera mis à l'honneur sur la page LinkedIn de l'enseignant. Le groupe pourra également y être mentionné avec l'accord de tous ses membres.

Amusez-vous et bon projet à toutes et à tous !

ANNEXE A — MODELE DE JOURNAL.MD

Copiez ce squelette à la racine de votre projet et complétez-le **au fil de l'eau**, pas la veille du dépôt.

```
# Journal — NomDuProjet (ALTN72, 2026-27)

**Groupe :** Prénom NOM, Prénom NOM

## README
- Prérequis : Java …, Maven …, base …
- Initialiser la base : `db/create.sql` (ou procédure …)
- Lancer : `mvn spring-boot:run`
- Application déployée : https://…
- Comptes de test : profil 1 : login / mot de passe ; profil 2 : …
- Limites connues : …

## ARCHITECTURE
- Dépôt : https://…
- Schéma : ![architecture](docs/architecture.png)

## TESTS
- Stratégie : …
- Commande : `mvn verify`
- Couvert : … | Non couvert : … (pourquoi)

## CHOIX TECHNIQUES
- Droits par profil : (tableau du §5.6)
- Exceptions : @ControllerAdvice ou ResponseStatusException ? Pourquoi ?
- SOLID : principe(s) appliqué(s) (où, pourquoi) ; principes écartés (pourquoi)
- Requête SQL native : où ? analyse critique
- @Transactional : où ? pourquoi ?

## LOT ALPHA
### Fonctionnalité transverse 1 : …
- Besoin · périmètre envisagé · choix et essais · ce qui fonctionne · difficultés · reste à faire
### Fonctionnalité transverse 2 : …

## BILAN
- Contribution de chacun : …
- Plus grande difficulté et comment elle a été surmontée : …
- Fonctionnalités non réalisées et pourquoi : …
- Trois points à retenir du cours et du projet : …
```

ANNEXE B — LISTE DE CONTROLE AVANT DEPOT

Cette liste est **à votre usage** : vous n'avez pas à la rendre. Elle vous évite les oublis les plus courants.

Fonctionnement

- ✓ Une copie **propre** du dépôt (nouveau dossier, nouveau clone) se lance en suivant uniquement le README.
- ✓ La base se crée et se remplit avec le script ou la procédure indiqués ; les données couvrent les cas limites (47 h, 48 h, 72 h…).
- ✓ Les deux profils peuvent se connecter avec les comptes indiqués ; chaque restriction est effective côté serveur.
- ✓ F1 à F12 sont opérationnelles et les règles RG1 à RG11 sont respectées.
- ✓ Au moins deux fonctionnalités transverses sont réalisées, démontrables et décrites.
- ✓ L'URL de l'application déployée fonctionne depuis un autre ordinateur.

Qualité

- ✓ `mvn verify` passe sans erreur sur la copie propre.
- ✓ Aucun import inutilisé, code commenté, `System.out.println` ou `printStackTrace`.
- ✓ Les seuils d'urgence existent à un seul endroit, sous forme de constantes nommées.
- ✓ Aucune règle métier dans les contrôleurs ou les vues.
- ✓ Les exceptions sont personnalisées, avec des messages en français ; aucun *catch* vide.
- ✓ Au moins un principe SOLID est documenté ; les autres sont justifiés.
- ✓ Les requêtes du repository combinent dérivées et JPQL, plus une seule requête SQL native analysée.
- ✓ Aucun mot de passe en clair, ni dans la base ni dans Git.

Livraison

- ✓ Le journal.md est complet : README, ARCHITECTURE, TESTS, CHOIX TECHNIQUES, LOT ALPHA, BILAN.
- ✓ L'historique Git montre un travail régulier de **chaque** membre du groupe.
- ✓ Chaque membre est capable d'expliquer n'importe quelle partie du code.
- ✓ Un seul dépôt par groupe est déposé sur Moodle avant le **5 novembre 2026 à 23 h 59**.

ANNEXE C — REFERENCES

Livres

- Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008 (en français : *Coder proprement*, Pearson).
- Martin Fowler, *Refactoring*, 2e édition, Addison-Wesley, 2018.
- Craig Walls, *Spring in Action*, 6e édition, Manning, 2022.
- Vladimir Khorikov, *Unit Testing Principles, Practices, and Patterns*, Manning, 2020.
- Scott Chacon, Ben Straub, *Pro Git*, 2e édition, Apress, 2014 (gratuit en ligne, en français).

Documentation officielle

- Ref liens dans le support de cours 😊

Conception, Clean Code et tests

- Refactoring.Guru, principes SOLID, patrons de conception, « code smells » : https://refactoring.guru/fr
- Baeldung, gestion des erreurs REST avec Spring : https://www.baeldung.com/exception-handling-for-rest-with-spring
- Martin Fowler, la pyramide de tests : https://martinfowler.com/bliki/TestPyramid.html
- JUnit 5 : https://junit.org/

Sécurité

- OWASP, Password Storage Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html